

Amazon RDS PostgreSQL IAM Authentication from EC2

To Begin with the Lab

Summary of the Lab

In this lab, IAM authentication was configured for an **Amazon RDS PostgreSQL** or **Aurora PostgreSQL-compatible** database and then tested from an **Amazon EC2** instance. The database instance was first modified to enable IAM database authentication, and an IAM user was created with an access key and an inline policy that mapped the database user to IAM access. A matching PostgreSQL database user was then created on the instance and granted the `rds_iam` role so it could authenticate through AWS IAM. On the EC2 instance, AWS CLI was configured with the IAM user credentials, an RDS authentication token was generated, and the SSL certificate bundle was downloaded. Finally, the EC2 host connected to PostgreSQL using `psql` with the temporary token and SSL, confirming that IAM-based authentication worked end to end.

Understanding IAM Authentication for PostgreSQL in Amazon RDS

IAM authentication lets you connect to PostgreSQL without using a permanent database password. Instead, you use an IAM identity to generate a temporary authentication token, and that token is passed to PostgreSQL along with SSL encryption. This exists so database access can be managed centrally through AWS permissions instead of storing passwords in multiple places. It is especially useful when you want temporary, auditable access from an EC2 bastion host or another AWS compute environment.

Example:

A Bastion Host needs short-term access to a PostgreSQL database for testing. The host uses an IAM user credential set to generate a token, then connects to the database with `psql` and the AWS RDS SSL bundle. The connection succeeds only while the token is valid, which is safer than using a fixed password.

Lab Steps

Step 1 – Enable IAM Authentication on the PostgreSQL Database

- Sign in to the AWS Management Console.
- Open **Amazon RDS**.
- Select the PostgreSQL DB instance or cluster.
- Click **Modify**.
- Scroll to the **Database authentication** section.

- Enable **IAM database authentication**.
- Click **Continue**.
- Select **Apply immediately**.
- Click **Modify DB instance** or **Modify cluster**.

Step 2 – Create the IAM User

- Open **IAM**.
- Click **Users**.

IAM Dashboard

Security recommendations

- Root user has MFA: Having multi-factor authentication (MFA) for the root user improves security for this account.
- Add MFA for yourself: Add multi-factor authentication (MFA) for yourself to improve security for this account.
- Your user, cf-shashank1, does not have any active access keys that have been unused for more than a year. Deactivating or deleting unused access keys improves security.

AWS Account

Account ID: 463646775279

Account Alias: Create

Sign-in URL for IAM users in this account: <https://463646775279.signin.aws.amazon.com/console>

IAM resources

Resources in this AWS Account

You are approaching your quota of available IAM resources. [Learn more](#)

User groups	Users	Roles	Policies	Identity providers
41	4	408	1128 372 available	1

Quick Links

- My security credentials
- Manage your access keys, multi-factor authentication (MFA) and other credentials.

Tools

- Online simulator

- Click **Create user**.

Managing human user access account by account? There's a better way.

Streamline human access to AWS and cloud apps when you enable Identity Center. [Learn more](#)

[Watch how it works](#)

- One-time set up for workforce user access
- Centrally manage access to multiple AWS accounts
- Provide access centrally to the cloud applications your workforce uses
- All with one-click access through a simple web portal

IAM users (4)

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

☐	User name	Path	Group	Last activity	MFA	Password age	Console last sign-in	Access key ID
☐	aws-aman	/	0	1 hour ago	-	3 days	1 hour ago	Active - AKIAWX44AK7...
☐	cf-shashank1	/	0	3 hours ago	-	3 days	3 hours ago	-
☐	cf-tushar	/	0	1 hour ago	-	2 days	1 hour ago	-
☐	Dave	/	1	2 days ago	-	2 days	2 days ago	-

- Enter a username such as `postgres_iam_user`.

Step 1: Specify user details

Step 2: Set permissions

Step 3: Review and create

Specify user details

User details

User name

postgres_iam_user

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and +, =, @, _ - (hyphen)

☐ Provide user access to the AWS Management Console - optional

In addition to console access, users with SignInLocalDevelopmentAccess permissions can use the same console credentials for programmatic access without the need for access keys.

[Learn more](#)

[If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user.](#)

[Cancel](#) [Next](#)

- Click **Next**.
- Create the user.
- Select the user and create an **access key**.

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

- Roles
- Policies
- IAM users**
 - IAM user groups
 - Identity providers
 - Account settings
 - Root access management
 - Temporary delegation requests
- Access reports**
 - Access Analyzer
 - Resource analysis
 - Unused access
 - Analyzer settings
 - Credential report
 - Organization activity
 - Service control policies
 - Resource control policies

postgres_iam_user [info](#) [Delete](#)

Summary

ARN arn:aws:iam::463646775279:user/postgres_iam_user	Console access Disabled	Access key 1 Create access key
Created June 22, 2026, 15:38 (UTC+05:30)	Last console sign-in -	

Permissions | Groups | Tags | Security credentials | Last Accessed

Permissions policies (0)

Permissions are defined by policies attached to the user directly or through groups.

Search Filter by Type All types

Policy name	Type	Attached via
No resources to display		

Permissions boundary (not set)

Generate policy based on CloudTrail events

- Use the **Command Line Interface (CLI)** use case.

aws

Search [Alt+S]

Global cloudriteshk (463646775279) cf-shashank1

IAM > IAM users > postgres-iam-user > Create access key

Step 1 - optional
Set description tag

Step 2 - optional
Set description tag

Step 3
Retrieve access keys

Use case

☒ Command Line Interface (CLI)
You plan to use this access key to enable application code running on an AWS compute service like Amazon EC2, Amazon ECS, or AWS Lambda to access your AWS account.

☐ Local code
You plan to use this access key to enable application code in a local development environment to access your AWS account.

☐ Application running on an AWS compute service
You plan to use this access key to enable application code running on an AWS compute service like Amazon EC2, Amazon ECS, or AWS Lambda to access your AWS account.

☐ Third-party service
You plan to use this access key to enable access for a third-party application or service that monitors or manages your AWS resources.

☐ Application running outside AWS
You plan to use this access key to authenticate workloads running in your data center or other infrastructure outside of AWS that needs to access your AWS resources.

☐ Other
Your use case is not listed here.

Alternatives recommended

- Use AWS CLI V2 and the `aws login` command to use your existing console credentials in the CLI. [Learn more](#)
- Use AWS CloudShell, a browser-based CLI, to run commands. [Learn more](#)

Confirmation

☒ I understand the above recommendation and want to proceed to create an access key.

- Save the access key and secret access key details in a safe place.

aws

Search [Alt+S]

Global cloudriteshk (463646775279) cf-shashank1

IAM > IAM users > postgres-iam-user > Create access key

ⓘ This is the only time that the secret access key can be viewed or downloaded. You cannot recover it later. However, you can create a new access key any time.

Step 1
Access key best practices & alternatives

Step 2 - optional
Set description tag

Step 3
Retrieve access keys

Retrieve access keys info

Access key
If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.

Access key
AKIAW44AK7XZ7D7F55

Secret access key
***** [Show](#)

Access key best practices

- Never store your access key in plain text, in a code repository, or in code.
- Disable or delete access key when no longer needed.
- Enable least-privilege permissions.
- Rotate access keys regularly.

For more details about managing access keys, see the [best practices for managing AWS access keys](#).

[Download.csv file](#) **Done**

Step 3 – Attach the IAM Policy

- Open the IAM user.
- Go to the **Permissions** tab.
- Click **Add inline policy**.

The screenshot shows the AWS IAM console for a user named 'postgres-iam-user'. The 'Permissions' tab is selected and highlighted with a red box. Below the summary, the 'Permissions policies (0)' section is visible. The 'Add permissions' button is highlighted with a red box, and a dropdown menu is open showing 'Add permissions' and 'Create inline policy', with 'Create inline policy' also highlighted with a red box.

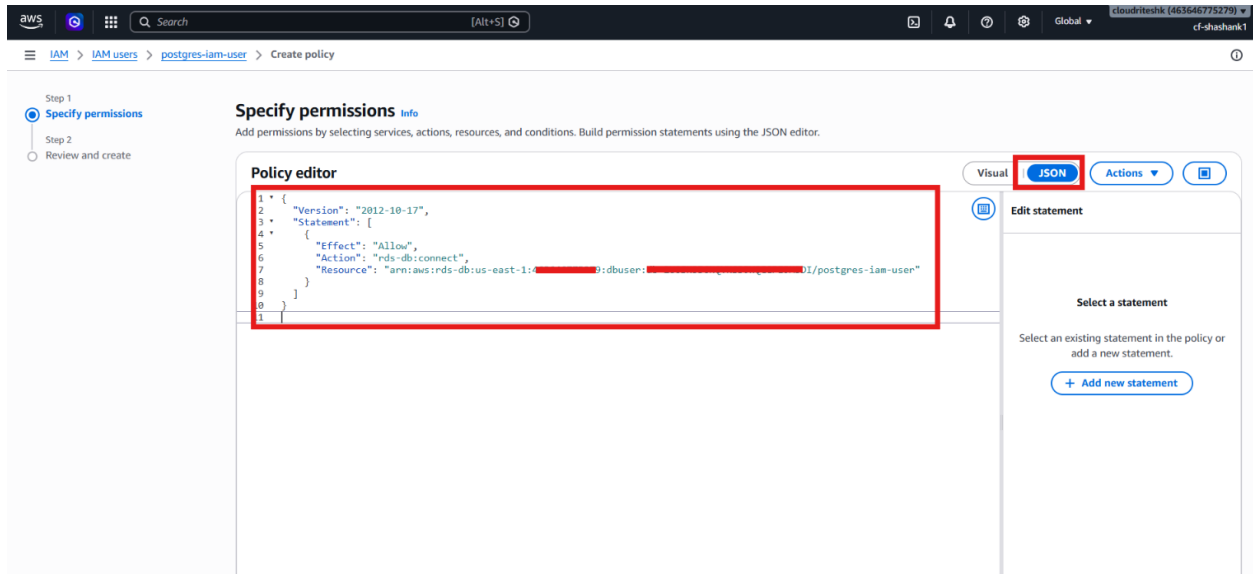
- Switch to the **JSON** tab.
- Add the policy that maps the IAM user to the RDS database user.
- You can find **Resource ID** in the **Configuration** Tab of your **Database**.

The screenshot shows the AWS RDS console for a database instance named 'database-1'. The 'Configuration' tab is selected and highlighted with a red box. In the 'Configuration' section, the 'Resource ID' field is highlighted with a red box, showing the value 'db-2UOSK3D6NQKSDJWQGGPEJM3DI'.

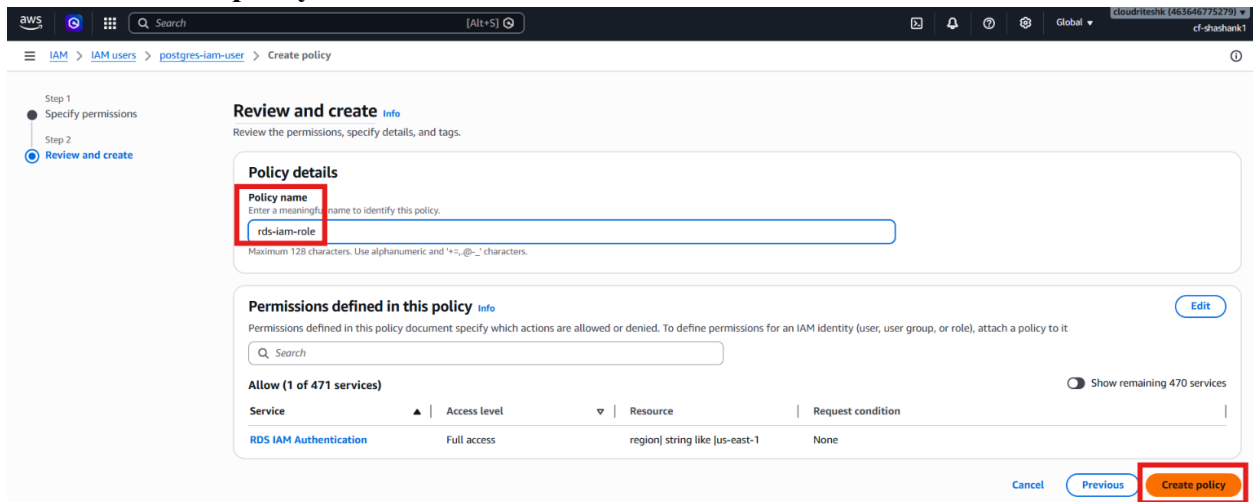
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "rds-db:connect",
      "Resource": "arn:aws:rds-db:<region>:<account-id>:dbuser:<db-resource-id>/<iam-db-user>"
    }
  ]
}
```

```
]
}
```

- Review the policy.

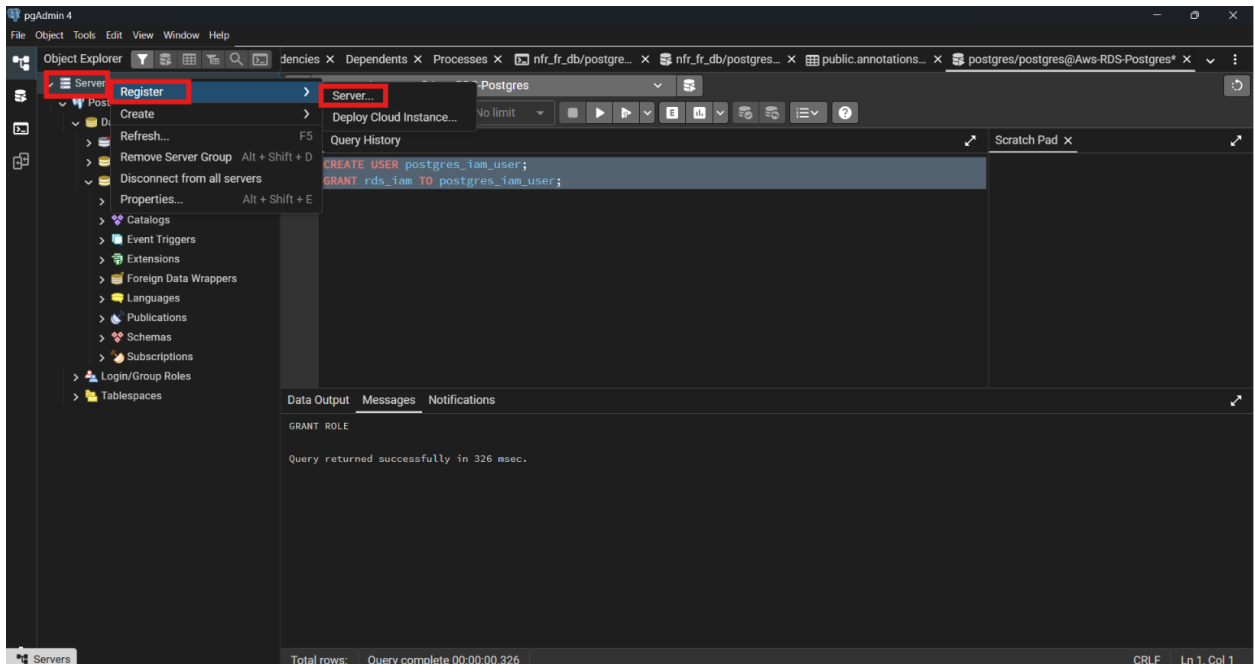


- Enter a policy name.
- Click **Create policy**.

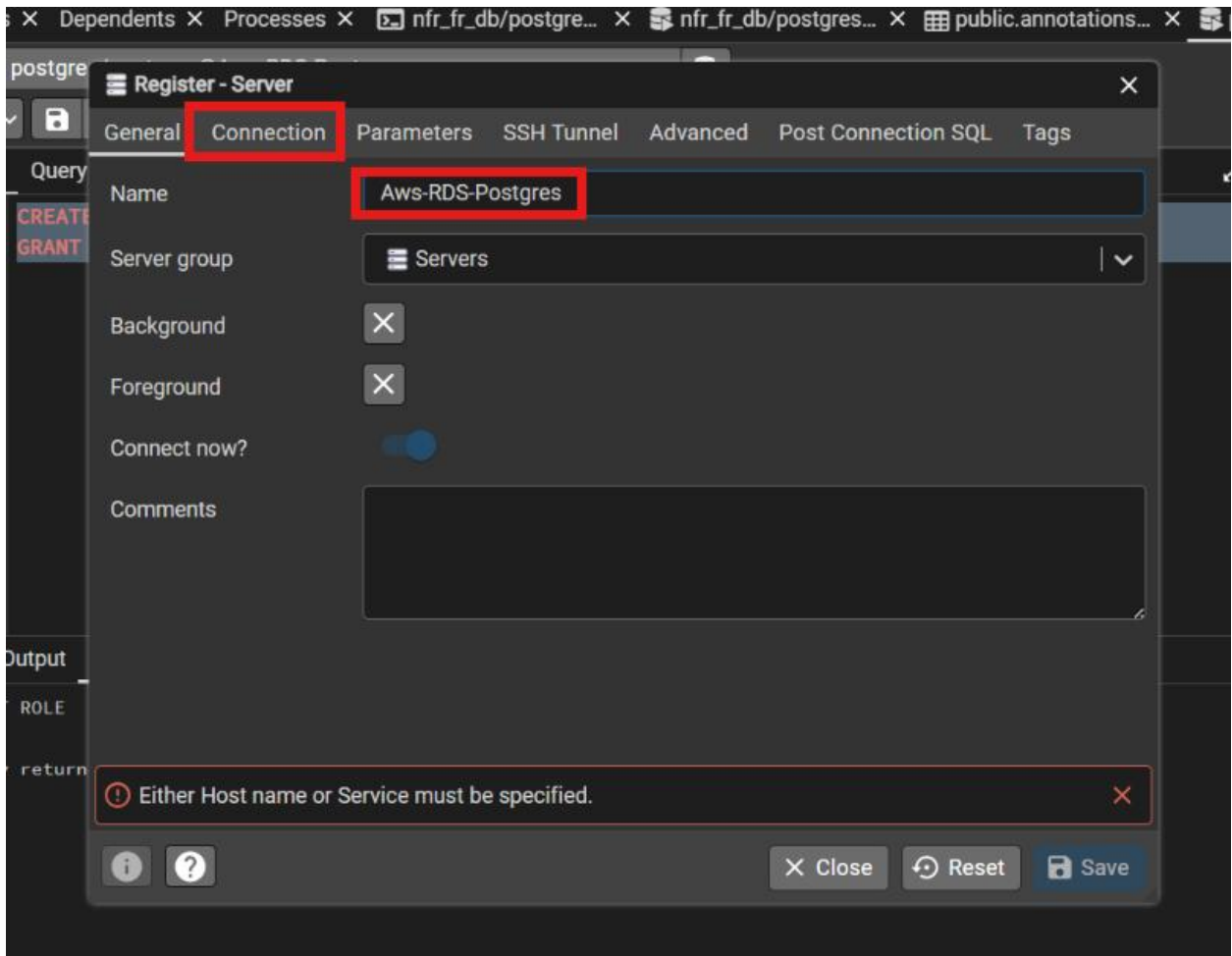


Step 4 – Connect to PostgreSQL Database using Master User

- Open pgAdmin and navigate to **Server**, right click and **Register** → **Server**.

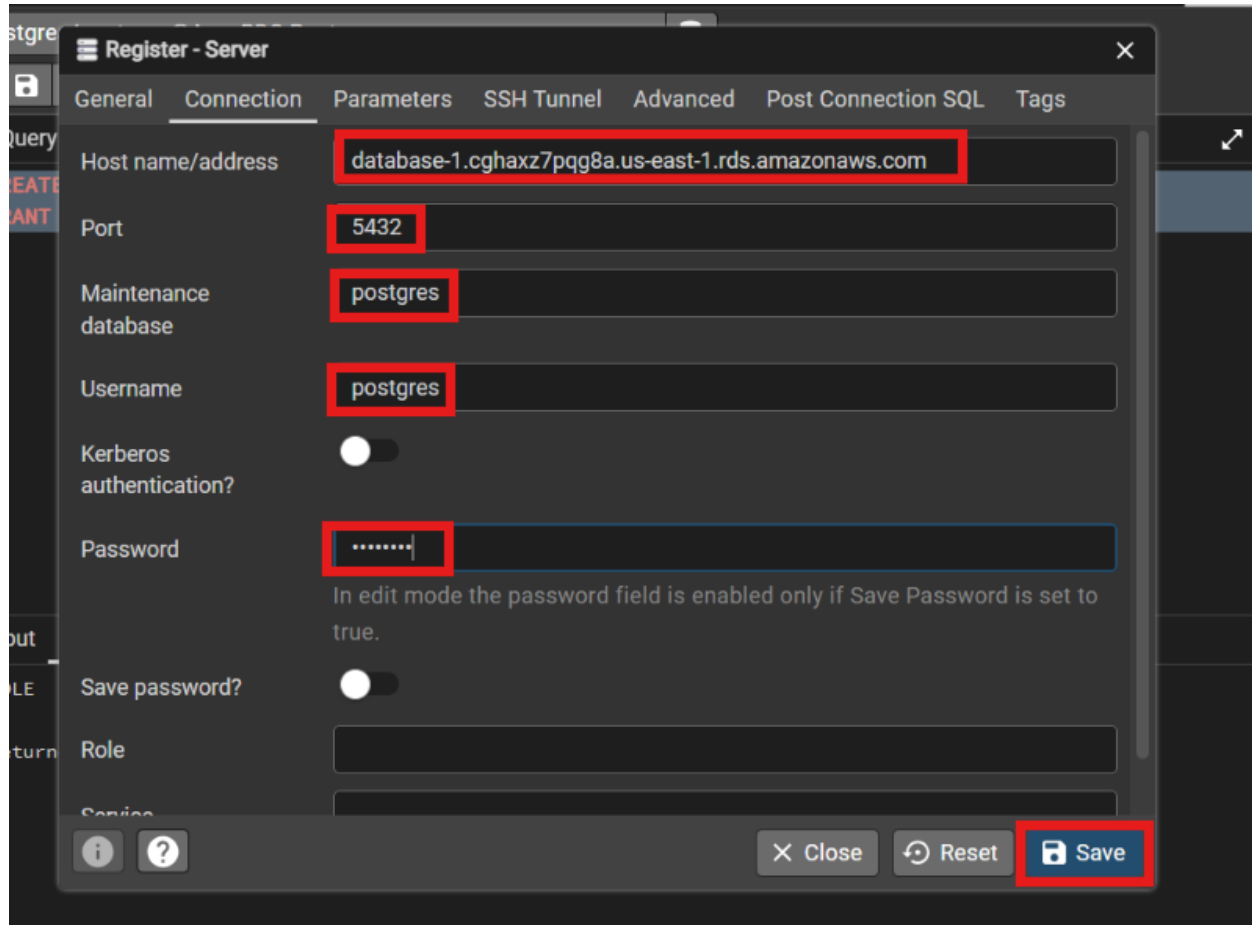


- Enter the name for **Server** and click on **Connections**.



- Enter the Required Details

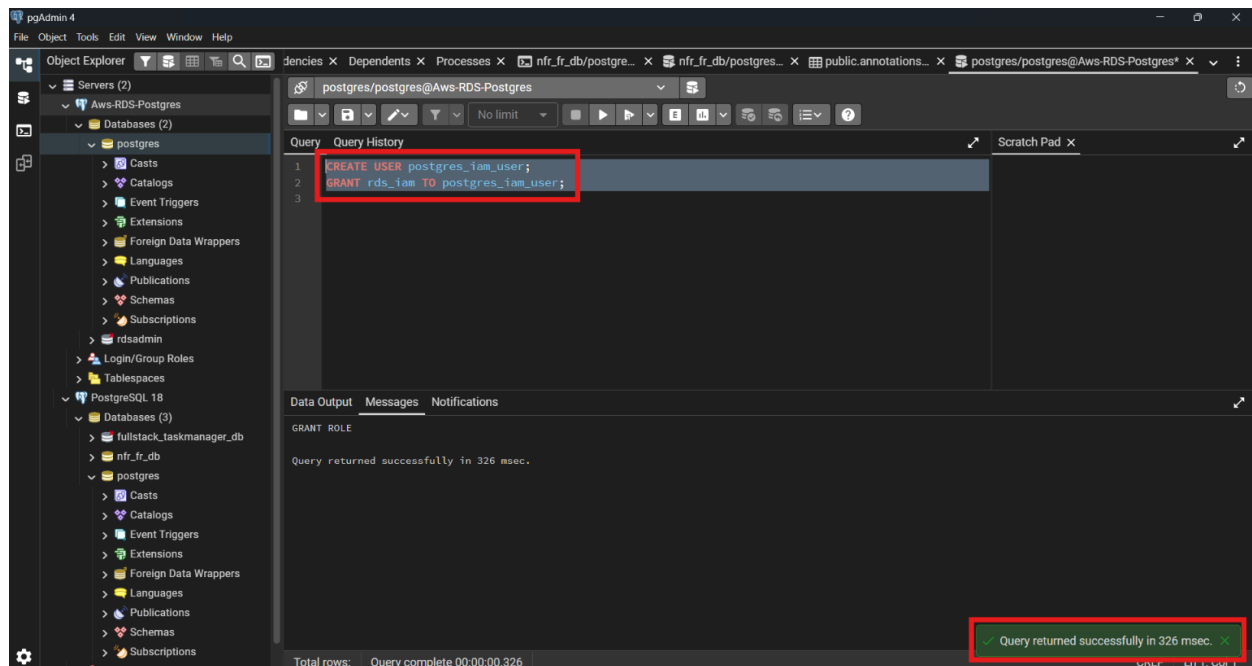
- Host name/address → Database Endpoint
- Port → 5432
- Username → Master username
- Password → Your master password
- And click **Save**.



Step 5 – Create the PostgreSQL Database User

- Create a database user with the same name as the IAM user.
- Grant the user the IAM authentication role.

```
CREATE USER iam_user;  
GRANT rds_iam TO iam_user;
```



- Confirm the database user is ready for IAM authentication.

Step 6 – Prepare the EC2 Instance

- Open **Amazon EC2**.
- Launch or open an existing **EC2** instance.
- Make sure to install **psql** on **EC2** Instance otherwise it will not run the command successfully.

```
sudo dnf install -y postgresql15
```

- Connect to the instance using SSH or Session Manager.
- Configure AWS CLI with the IAM user credentials.

```
aws configure
```

- Enter the access key, secret access key, default region, and output format.


```
ec2-user@ip-172-31-30-102:~$ export RDSHOST="database-1.cghaxz7pgg8a.us-east-1.rds.amazonaws.com"
ec2-user@ip-172-31-30-102:~$ export PGPASSWORD="$(aws rds generate-db-auth-token --hostname $RDSHOST --port 5432 --region us-east-1 --username postgres)"
ec2-user@ip-172-31-30-102:~$ echo $PGPASSWORD
database-1.cghaxz7pgg8a.us-east-1.rds.amazonaws.com:5432/?Action=connect&DBUser=postgres&X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKIAW44AK7X7Z7D7F5S%2F20260622%2Fus-east-1%2Frds-db%2Faws4_request&X-Amz-Date=20260622T102931Z&X-Amz-Expires=900&X-Amz-SignedHeaders=host&X-Amz-Signature=a2231513cb1987125a69e19b70dc96a00cce3cbbf433262e6385954688296b0d
ec2-user@ip-172-31-30-102:~$ wget https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
--2026-06-22 10:31:08-- https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
Resolving truststore.pki.rds.amazonaws.com (truststore.pki.rds.amazonaws.com)... 13.226.209.110, 13.226.209.117, 13.226.209.35, ...
Connecting to truststore.pki.rds.amazonaws.com (truststore.pki.rds.amazonaws.com)|13.226.209.110|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 165408 (162K) [binary/octet-stream]
Saving to: 'global-bundle.pem'

global-bundle.pem      100%[=====] 161.53K  --.-KB/s  in 0.002s

2026-06-22 10:31:08 (71.9 MB/s) - 'global-bundle.pem' saved [165408/165408]

ec2-user@ip-172-31-30-102:~$ psql -h $RDSHOST -p 5432 "sslmode=verify-full sslrootcert=./global-bundle.pem dbname=postgres user=postgres_iam_user"
-bash: psql: command not found
ec2-user@ip-172-31-30-102:~$ sudo dnf install -y postgresql
Amazon Linux 2023 Kernel Livepatch repository                               391 kB/s | 48 kB    00:00
No match for argument: postgresql
Error: Unable to find a match: postgresql
ec2-user@ip-172-31-30-102:~$ sudo dnf install -y postgresql15
Last metadata expiration check: 0:00:35 ago on Mon Jun 22 10:36:21 2026.
Dependencies resolved.
```

- Use the token as the temporary database password.

Step 8 – Download the SSL Certificate Bundle

- Download the AWS RDS SSL certificate bundle on the EC2 instance using this command.

```
wget https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
```

```
ec2-user@ip-172-31-30-102:~$ export RDSHOST="database-1.cghaxz7pgg8a.us-east-1.rds.amazonaws.com"
ec2-user@ip-172-31-30-102:~$ export PGPASSWORD="$(aws rds generate-db-auth-token --hostname $RDSHOST --port 5432 --region us-east-1 --username postgres)"
ec2-user@ip-172-31-30-102:~$ echo $PGPASSWORD
database-1.cghaxz7pgg8a.us-east-1.rds.amazonaws.com:5432/?Action=connect&DBUser=postgres&X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKIAW44AK7X7Z7D7F5S%2F20260622%2Fus-east-1%2Frds-db%2Faws4_request&X-Amz-Date=20260622T102931Z&X-Amz-Expires=900&X-Amz-SignedHeaders=host&X-Amz-Signature=a2231513cb1987125a69e19b70dc96a00cce3cbbf433262e6385954688296b0d
ec2-user@ip-172-31-30-102:~$ wget https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
--2026-06-22 10:31:08-- https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
Resolving truststore.pki.rds.amazonaws.com (truststore.pki.rds.amazonaws.com)... 13.226.209.110, 13.226.209.117, 13.226.209.35, ...
Connecting to truststore.pki.rds.amazonaws.com (truststore.pki.rds.amazonaws.com)|13.226.209.110|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 165408 (162K) [binary/octet-stream]
Saving to: 'global-bundle.pem'

global-bundle.pem      100%[=====] 161.53K  --.-KB/s  in 0.002s

2026-06-22 10:31:08 (71.9 MB/s) - 'global-bundle.pem' saved [165408/165408]

ec2-user@ip-172-31-30-102:~$ psql -h $RDSHOST -p 5432 "sslmode=verify-full sslrootcert=./global-bundle.pem dbname=postgres user=postgres_iam_user"
-bash: psql: command not found
ec2-user@ip-172-31-30-102:~$ sudo dnf install -y postgresql
Amazon Linux 2023 Kernel Livepatch repository                               391 kB/s | 48 kB    00:00
No match for argument: postgresql
Error: Unable to find a match: postgresql
ec2-user@ip-172-31-30-102:~$ sudo dnf install -y postgresql15
Last metadata expiration check: 0:00:35 ago on Mon Jun 22 10:36:21 2026.
Dependencies resolved.
```

- Save the certificate bundle locally on the host.
- Use the certificate bundle for secure PostgreSQL connections.

Step 9 – Connect to PostgreSQL with IAM Authentication

- Use psql to connect to the database.

- Pass the IAM token as the password.
- Include SSL certificate validation when connecting.

```
psql "host=$RDSHOST port=5432 dbname=<DATABASE_NAME> user=<DB_USER>
sslmode=verify-full sslrootcert=./global-bundle.pem"
```

- Verify the session connection information.

\conninfo

```
ec2-user@ip-172-31-30-102:~$ sudo yum install postgresql15-private-libs-15.18-1.amzn2023.0.1.x86_64
Preparing : postgresql15-private-libs-15.18-1.amzn2023.0.1.x86_64 1/1
Installing : postgresql15-private-libs-15.18-1.amzn2023.0.1.x86_64 1/2
Installing : postgresql15-15.18-1.amzn2023.0.1.x86_64 2/2
Running scriptlet: postgresql15-15.18-1.amzn2023.0.1.x86_64 2/2
Verifying : postgresql15-15.18-1.amzn2023.0.1.x86_64 1/2
Verifying : postgresql15-private-libs-15.18-1.amzn2023.0.1.x86_64 2/2

Installed:
  postgresql15-15.18-1.amzn2023.0.1.x86_64      postgresql15-private-libs-15.18-1.amzn2023.0.1.x86_64

Complete!
[ec2-user@ip-172-31-30-102 ~]$ psql -h $RDSHOST -p 5432 "sslmode=verify-full sslrootcert=./global-bundle.pem dbname=postgres user=postgres_iam_user"
psql: error: could not translate host name "-p" to address: Name or service not known
[ec2-user@ip-172-31-30-102 ~]$ export RDSHOST="database-1.cghaxz7pqq8a.us-east-1.rds.amazonaws.com"
[ec2-user@ip-172-31-30-102 ~]$ export PGPASSWORD="$(aws rds generate-db-auth-token --hostname $RDSHOST --port 5432 --region us-east-1 --username postgres_iam_user)"
[ec2-user@ip-172-31-30-102 ~]$ psql "host=$RDSHOST port=5432 dbname=postgres user=postgres_iam_user sslmode=verify-full sslrootcert=./global-bundle.pem"
psql (15.10, server 15.2)
WARNING: psql major version 15, server major version 18.
Some psql features might not work.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
Type "help" for help.

postgres=> \conninfo
You are connected to database "postgres" as user "postgres_iam_user" on host "database-1.cghaxz7pqq8a.us-east-1.rds.amazonaws.com" (address "172.31.77.56") at port "5432".
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
postgres=> |
```

- Confirm that the session is authenticated as the IAM database user.

Step 10 – Clean Up

- Remove the IAM inline policy from the user.
- Delete the IAM user if it is no longer needed.
- Remove any temporary scripts or tokens from the EC2 host.
- Disable IAM authentication on the database if the lab is complete.
- Delete the lab resources to avoid charges.

Verification

- IAM authentication was enabled on the PostgreSQL database.
- The IAM user was created and given an access key.
- The IAM inline policy mapped the IAM user to the database user.
- The PostgreSQL database user was created with the same name as the IAM user.
- The EC2 instance successfully generated an RDS authentication token.
- The PostgreSQL connection worked with psql using IAM authentication and SSL.
- The lab confirmed end-to-end IAM-based database access.